



Universidad Tecmilenio  
Estructuras de Datos  
Avance de Proyecto Final  
Canek Lugo Soto  
2967116

## Avance del Proyecto

### Clase Cancion

```
1  import java.util.*;
2
3  class Cancion {
4      private String nombre;
5      private String genero;
6
7      public Cancion(String nombre, String genero) {
8          this.nombre = nombre;
9          this.genero = genero;
10     }
11
12     public String getNombre() { return nombre; }
13     public String getGenero() { return genero; }
14
15     @Override
16     public String toString() {
17         return nombre + " (" + genero + ")";
18     }
19 }
```

La clase Cancion se encarga de representar cada canción del reproductor. Guarda el nombre y el género, y nos permite mostrar la información de forma ordenada cuando se imprime.

### Pila

```
21 class Pila {
22     private Stack<Cancion> pila = new Stack<>();
23
24     public void push(Cancion c) { pila.push(c); }
25     public Cancion pop() { return pila.isEmpty() ? null : pila.pop(); }
26     public Cancion peek() { return pila.isEmpty() ? null : pila.peek(); }
27     public List<Cancion> mostrar() {
28         List<Cancion> lista = new ArrayList<>(pila);
29         Collections.reverse(lista); // Para mostrar en orden
30         return lista;
31     }
32 }
```

La pila funciona con la lógica LIFO. En el programa se representa una playlist donde las canciones se van apilando y se reproducen en orden inverso. Esto puede servir para escuchar lo más nuevo que se ha agregado a dicha playlist de canciones.

## Cola

```
34 class Cola {
35     private Queue<Cancion> cola = new LinkedList<>();
36
37     public void enqueue(Cancion c) { cola.add(c); }
38     public Cancion dequeue() { return cola.poll(); }
39     public Cancion front() { return cola.peek(); }
40     public List<Cancion> mostrar() { return new ArrayList<>(cola); }
41 }
```

La cola trabaja con la lógica FIFO. En este caso se representan las canciones en espera, que se reproducen en el orden en que fueron agregadas.

## Lista

```
44 class Lista {
45     private List<Cancion> lista = new ArrayList<>();
46
47     public void insertar(Cancion c) { lista.add(c); }
48     public void eliminar(String nombre) {
49         lista.removeIf(c -> c.getNombre().equalsIgnoreCase(nombre));
50     }
51     public List<Cancion> buscar(String nombre) {
52         List<Cancion> encontradas = new ArrayList<>();
53         for (Cancion c : lista) {
54             if (c.getNombre().equalsIgnoreCase(nombre)) {
55                 encontradas.add(c);
56             }
57         }
58         return encontradas;
59     }
60     public List<Cancion> mostrar() { return lista; }
61 }
```

La lista permite guardar canciones de manera flexible. A diferencia de la pila o la cola, aquí puedes insertar, eliminar o buscar canciones en cualquier momento. Es como tener una playlist personalizada que puedes editar libremente sin importar el orden en que agregaste los temas.

## Menú

```
63 public class ReproductorMusica {
    Run main | Debug main | Run | Debug
64     public static void main(String[] args) {
65         Scanner sc = new Scanner(System.in);
66         Pila pila = new Pila();
67         Cola cola = new Cola();
68         Lista lista = new Lista();
69
70         while (true) {
71             System.out.println(x:"\n--- REPRODUCTOR ---");
72             System.out.println(x:"1. Agregar a Playlist (Pila)");
73             System.out.println(x:"2. Reproducir de Playlist (Pila)");
74             System.out.println(x:"3. Agregar a Cola de espera (Cola)");
75             System.out.println(x:"4. Reproducir de Cola de espera (Cola)");
76             System.out.println(x:"5. Agregar a Lista");
77             System.out.println(x:"6. Eliminar de Lista");
78             System.out.println(x:"7. Buscar en Lista");
79             System.out.println(x:"8. Ver todas las canciones ordenadas");
80             System.out.println(x:"9. Salir");
81             System.out.print(s:"Elige una opción: ");
82             String opcion = sc.nextLine();
83
84             switch (opcion) {
85                 case "1":
86                     System.out.print(s:"Nombre: ");
87                     String n1 = sc.nextLine();
88                     System.out.print(s:"Género: ");
89                     String g1 = sc.nextLine();
90                     pila.push(new Cancion(n1, g1));
91                     break;
92
93                 case "2":
94                     Cancion c1 = pila.pop();
95                     System.out.println("Reproduciendo: " + (c1 != null ? c1 : "Nada"));
96                     break;
97
98                 case "3":
99                     System.out.print(s:"Nombre: ");
100                    String n2 = sc.nextLine();
101                    System.out.print(s:"Género: ");
102                    String g2 = sc.nextLine();
103                    cola.enqueue(new Cancion(n2, g2));
104                    break;
105
106                 case "4":
107                     Cancion c2 = cola.dequeue();
108                     System.out.println("Reproduciendo: " + (c2 != null ? c2 : "Nada"));
109                     break;
```

```
111 case "5":
112     System.out.print(s:"Nombre: ");
113     String n3 = sc.nextLine();
114     System.out.print(s:"Género: ");
115     String g3 = sc.nextLine();
116     lista.insertar(new Cancion(n3, g3));
117     break;
118
119 case "6":
120     System.out.print(s:"Nombre a eliminar: ");
121     String elim = sc.nextLine();
122     lista.eliminar(elim);
123     break;
124
125 case "7":
126     System.out.print(s:"Nombre a buscar: ");
127     String busc = sc.nextLine();
128     System.out.println("Encontradas: " + lista.buscar(busc));
129     break;
130
131 case "8":
132     List<Cancion> todas = new ArrayList<>();
133     todas.addAll(pila.mostrar());
134     todas.addAll cola.mostrar());
135     todas.addAll(lista.mostrar());
136
```

```

137  if (!todas.isEmpty()) {
138      System.out.println(x:"Ordenadas alfabéticamente:");
139      todas.stream()
140          .sorted(Comparator.comparing(Cancion::getNombre))
141          .forEach(System.out::println);
142
143      System.out.println(x:"\nOrdenadas por género:");
144      todas.stream()
145          .sorted(Comparator.comparing(Cancion::getGenero))
146          .forEach(System.out::println);
147  } else {
148      System.out.println(x:"No hay canciones registradas");
149  }
150  break;
151
152  case "9":
153      System.out.println(x:"Saliendo...");
154      sc.close();
155      return;
156
157  default:
158      System.out.println(x:"Opción no válida.");
159  }
160 }
161 }
162 }
163

```

El menú es la parte interactiva del programa, lo que permite al usuario elegir qué acción realizar. Desde aquí se controlan las opciones para agregar, reproducir, eliminar o buscar canciones. Funciona como la pantalla principal de un reproductor de música real, con botones para cada opción.

Finalmente así se ve el menú del reproductor:

```

PS C:\Users\canek\OneDrive - Universidad Tecmilenio\Estructura de Datos> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\canek\AppData\Roaming\Code\User\workspaceStorage\44731a53df0c249874078943b480b00f\redhat.java\jdt_ws\Estructura de Datos_94088da9\bin' 'ReproductorMusica'

--- REPRODUCTOR ---
1. Agregar a Playlist (Pila)
2. Reproducir de Playlist (Pila)
3. Agregar a Cola de espera (Cola)
4. Reproducir de Cola de espera (Cola)
5. Agregar a Lista
6. Eliminar de Lista
7. Buscar en Lista
8. Ver todas las canciones ordenadas
9. Salir
Elige una opción: █

```